

PROGRAMACIÓN III

Resumen de Estudio

TP5 al TP8 + 2do Parcial

toString · equals · hashCode · Lombok · DTOs Programación Funcional · Streams · JPA · Hibernate
Repositorios Genéricos · JPQL · Menú ABM

Patricio María Sussini Guanziroli

UTN · Tecnicatura Universitaria en Programación · 2026

Índice de contenidos

TP 5	toString, equals y hashCode	Implementación manual de los tres contratos fundamentales del objeto Java. Uso con HashSet y HashMap.
TP 6	Lombok y DTOs	Eliminación de boilerplate con anotaciones Lombok. Patrón SuperBuilder. Data Transfer Objects con Java Records.
TP 7	Programación Funcional — Streams	Lambdas, interfaces funcionales, operaciones intermedias y terminales, Collectors, Optional.
TP 8	JPA con Hibernate	Jakarta Persistence API, mapeo de entidades, relaciones ManyToOne/OneToMany, ciclo de vida, JPQL.
2do Parcial	Repositorios Genéricos y Menú ABM	Patrón Repository genérico, BaseRepository, CategoriaRepository, ProductoRepository, menú de consola.
Guión	Presentación del Video	Estructura sugerida para grabar el video de presentación del 2do parcial.

¿Por qué existen estos tres métodos?

En Java, toda clase hereda de **Object**. Object define por defecto tres métodos críticos: **toString()**, **equals()** y **hashCode()**. El problema es que las implementaciones por defecto comparan referencias de memoria, no contenido. Para que dos objetos con los mismos datos sean considerados iguales (en colecciones, búsquedas, etc.) hay que sobrescribir estos métodos.

toString()

Convierte el objeto a una representación legible en texto. Java lo llama automáticamente al concatenar un objeto con un String o al pasarlo a `println()`. Sin sobrescribir, imprime algo como: **Producto@3e25a5** (clase + dirección de memoria).

Ejemplo — toString() manual

```
@Override
public String toString() {
    return "Producto{nombre='" + nombre + "', precio=" + precio + "}";
}
```

equals()

Define cuándo dos objetos son *equivalentes* según el negocio. Por defecto compara referencias (`==`). Al sobrescribir, comparamos campos significativos.

Contrato de equals() — obligatorio cumplir:

- **Reflexivo:** `x.equals(x)` siempre true.
- **Simétrico:** `x.equals(y) ↔ y.equals(x)`.
- **Transitivo:** si `x==y` e `y==z`, entonces `x==z`.
- **Consistente:** mismo resultado si el objeto no cambia.
- **Null-safe:** `x.equals(null)` siempre false.

Ejemplo — equals() en Producto (por nombre)

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Producto)) return false;
    Producto p = (Producto) o;
    return Objects.equals(nombre, p.nombre);
}
```

hashCode()

Devuelve un número entero que identifica al objeto para estructuras basadas en hashing (**HashSet**, **HashMap**, **Hashtable**). El motor usa hashCode() para ubicar el bucket y luego equals() para confirmar.

■ **Regla de oro: si dos objetos son iguales según equals(), DEBEN tener el mismo hashCode(). Si se rompe esta regla, HashSet y HashMap fallan silenciosamente.**

Ejemplo — hashCode() consistente con equals()

```
@Override
public int hashCode() {
    return Objects.hash(nombre); // mismos campos que equals()
}
```

Cómo funciona HashSet internamente

Cuando hacemos **set.add(objeto)** o **set.contains(objeto)**:

Paso 1	Calcula hashCode() del objeto → determina el bucket (posición en el array interno).
Paso 2	Dentro del bucket, recorre los elementos y aplica equals() para evitar duplicados.
Paso 3	Si equals() devuelve true → considera que ya existe → no inserta.

¿Qué es Lombok?

Lombok es una librería que genera automáticamente en tiempo de compilación el código repetitivo (boilerplate): getters, setters, constructores, equals, hashCode, toString. El código generado no aparece en los archivos .java pero sí en los .class compilados.

Anotaciones principales

Anotación	Descripción
@Getter / @Setter	Genera get/set para todos los campos (o uno específico si se pone en el campo).
@NoArgsConstructor	Genera constructor sin parámetros. Requerido por JPA.
@AllArgsConstructor	Genera constructor con todos los campos como parámetros.
@ToString	Genera toString() con todos los campos. Con callSuper=true incluye los de la superclase.
@EqualsAndHashCode	Genera equals() y hashCode(). Con callSuper=true incluye campos del padre. Con onlyExplicitlyIncluded=true solo usa los marcados con @Include.
@Builder	Patrón Builder para la clase. No funciona bien con herencia.
@SuperBuilder	Versión de Builder que soporta herencia. Requiere anotarlo en la clase padre e hijas.

@SuperBuilder — Patrón Builder con herencia

Permite construir objetos con una cadena fluida de métodos en jerarquías de herencia. Se debe anotar TANTO la clase padre (Base) como cada clase hija.

Ejemplo — SuperBuilder en acción

```
// Construcción con builder
Producto p = Producto.builder()
    .nombre("Notebook")           // campo de Producto
    .precio(1500.0)
    .eliminado(false)             // campo de Base (superclase)
    .createdAt(LocalDateTime.now())
    .build();
```

DTOs — Data Transfer Objects

Un DTO es un objeto que transporta solo los datos necesarios, ocultando información sensible (contraseña, rol) o simplificando la estructura. Se crea a partir de una entidad pero no es una entidad JPA.

Java Record como DTO

Desde Java 16, los **Records** son clases inmutables con constructor, equals, hashCode y toString generados automáticamente. Son ideales para DTOs porque son concisos y no permiten modificar su estado tras la construcción.

Ejemplo — UsuarioDTO como record

```
public record UsuarioDTO(  
    Long id,  
    String nombre,  
    String apellido,  
    String mail  
  
    // SIN contraseña ni rol  
) {}  
  
// Uso:  
  
UsuarioDTO dto = new UsuarioDTO(usuario.getId(),  
    usuario.getNombre(), usuario.getApellido(), usuario.getMail());
```

¿Qué es la Programación Funcional?

Es un paradigma que trata la computación como evaluación de funciones matemáticas. En Java (desde Java 8), se incorpora mediante **Lambdas**, **Streams** e **Interfaces Funcionales**. El objetivo es escribir código más declarativo (qué hacer) y menos imperativo (cómo hacerlo).

Lambdas

Una lambda es una función anónima que se puede pasar como argumento. Su sintaxis es: **(parámetros)** -> **cuerpo**

Ejemplos de lambda

```
// Lambda con un parametro
productos.forEach(p -> System.out.println(p.getNombre()));

// Lambda como predicado
Predicate<Producto> disponible = p -> p.getDisponible();

// Referencia a metodo (azucar sintactico de lambda)
productos.forEach(System.out::println);
```

Interfaces Funcionales principales

Interfaz	Firma / Ejemplo
Predicate<T>	$T \rightarrow \text{boolean}$ Ej: Filtrar: $p \rightarrow p.getStock() > 0$
Function<T,R>	$T \rightarrow R$ Ej: Transformar: $p \rightarrow p.getNombre()$
Consumer<T>	$T \rightarrow \text{void}$ Ej: Consumir: $p \rightarrow \text{System.out.println}(p)$
Supplier<T>	$() \rightarrow T$ Ej: Proveer: $() \rightarrow \text{new Producto}()$
Comparator<T>	$(T, T) \rightarrow \text{int}$ Ej: Ordenar: $\text{Comparator.comparing}(\text{Producto}::\text{getPrecio})$

Streams — flujo de datos

Un Stream es una secuencia de elementos que soporta operaciones encadenadas. **No modifica la colección original** y puede ser secuencial o paralelo.

Operaciones Intermedias — devuelven otro Stream (lazy)

Operación	Qué hace
filter(Predicate)	Descarta elementos que no cumplen la condición.
map(Function)	Transforma cada elemento en otro.

sorted(Comparator)	Ordena el stream.
distinct()	Elimina duplicados (usa equals()).
limit(n)	Toma solo los primeros n elementos.
peek(Consumer)	Inspecciona sin transformar (útil para debug).

Operaciones Terminales — consumen el Stream (eager)

Operación	Qué hace
forEach(Consumer)	Ejecuta acción por cada elemento.
collect(Collector)	Acumula en colección (List, Set, Map).
count()	Cuenta los elementos.
findFirst()	Devuelve Optional con el primer elemento.
anyMatch(Predicate)	True si alguno cumple la condición.
reduce(identity, BinaryOp)	Reduce a un solo valor (suma, concatenar, etc.).

Ejemplo completo con el dominio del TP

Streams sobre listas de Producto
<pre>// Filtrar productos disponibles con stock bajo List<Producto> stockBajo = productos.stream() .filter(Producto::getDisponible) .filter(p -> p.getStock() < 5) .collect(Collectors.toList()); // Calcular total de un pedido double total = detalles.stream() .mapToDouble(d -> d.getPrecio() * d.getCantidad()) .sum(); // Agrupar productos por categoria Map<String, List<Producto>> porCat = productos.stream() .collect(Collectors.groupingBy(p -> p.getCategoria().getNombre()));</pre>

Optional

Contenedor que puede o no tener un valor. Evita NullPointerException al forzar al desarrollador a manejar el caso de ausencia explícitamente.

Uso de Optional
<pre>Optional<Producto> opt = repo.buscarPorId(id); // Verificar presencia</pre>


```
if (opt.isPresent()) { ... }  
if (opt.isEmpty()) { ... }  
  
// Obtener valor (lanza excepcion si vacio)  
Producto p = opt.get();  
  
// Obtener o default  
Producto p = opt.orElse(new Producto());  
  
// Obtener o lanzar excepcion personalizada  
Producto p = opt.orElseThrow(() -> new RuntimeException("No encontrado));
```

¿Qué es JPA?

JPA (Jakarta Persistence API) es una especificación de Java para mapear objetos a tablas de bases de datos relacionales (**ORM — Object Relational Mapping**). **Hibernate** es la implementación más popular de JPA. Con JPA, trabajamos con objetos Java y Hibernate se encarga de generar y ejecutar el SQL.

Anotaciones JPA esenciales

Anotación	Descripción
@Entity	Marca la clase como entidad persistente (tabla en la BD).
@Table(name=...)	Especifica el nombre de la tabla. Por defecto usa el nombre de la clase.
@MappedSuperclass	La clase no tiene tabla propia, pero sus campos se heredan. Usada en Base.
@Id	Marca el campo como clave primaria.
@GeneratedValue(IDENTITY)	El ID lo genera la BD automáticamente (autoincrement).
@Column(unique=true)	Mapea un campo a una columna. unique=true agrega constraint.
@ManyToOne	Relación muchos-a-uno. Ej: muchos Productos → una Categoría.
@OneToMany	Relación uno-a-muchos. Ej: un Usuario → muchos Pedidos.
@JoinColumn(name=...)	Define la columna de clave foránea en la tabla.
@Enumerated(STRING)	Persiste un enum como texto en la BD (no como número).

Clase Base con @MappedSuperclass

Patrón donde una clase abstracta tiene los campos comunes (id, eliminado, createdAt) y todas las entidades la extienden. No genera tabla propia.

Base.java

```
@Getter @Setter @SuperBuilder @NoArgsConstructor @AllArgsConstructor
@MappedSuperclass
public abstract class Base {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private boolean eliminado;
```

```
private LocalDateTime createdAt;  
}
```

FetchType — LAZY vs EAGER

Define *cuándo* se carga una relación desde la base de datos:

- **EAGER**: se carga junto con la entidad principal en el mismo SELECT. Conveniente pero puede traer datos de más.
- **LAZY**: se carga solo cuando se accede al campo por primera vez (proxy). Más eficiente pero requiere que el EntityManager esté abierto.

■ *Por defecto: @ManyToOne es EAGER. @OneToMany es LAZY.*

CascadeType y orphanRemoval

CascadeType	Efecto
PERSIST	Al persistir la entidad padre, se persisten las hijas también.
MERGE	Al hacer merge del padre, se mergean las hijas.
REMOVE	Al eliminar el padre, se eliminan las hijas.
ALL	Aplica todos los anteriores.

orphanRemoval=true: si se saca un hijo de la colección del padre, se elimina automáticamente de la BD (queda huérfano).

EntityManager — operaciones principales

Método	Descripción
persist(obj)	Inserta un objeto nuevo (transient → managed).
merge(obj)	Actualiza un objeto o persiste uno nuevo si no tiene ID. Devuelve la instancia managed.
find(Clase, id)	Busca por clave primaria. Devuelve null si no existe.
remove(obj)	Elimina el objeto de la BD (debe estar managed).
createQuery(jpql, Clase)	Crea una TypedQuery para ejecutar JPQL.

JPQL — Java Persistence Query Language

JPQL es como SQL pero trabaja sobre **entidades y campos Java**, no sobre tablas y columnas. Es portable entre distintos motores de base de datos.

Ejemplos JPQL

```
// Listar activos (usando nombre de la entidad Java, no la tabla)  
SELECT e FROM Categoria e WHERE e.eliminado = false  
  
// Buscar por campo de entidad relacionada (NO necesita JOIN explícito)  
SELECT p FROM Producto p WHERE p.categoria.id = :categoriaId
```

```
AND p.eliminado = false
```

```
// Uso con TypedQuery (sin casteos, type-safe)
```

```
TypedQuery<Producto> q = em.createQuery(jpql, Producto.class);
```

```
q.setParameter("categoriaId", id);
```

```
List<Producto> resultado = q.getResultList();
```

persistence.xml — configuración central

Archivo en **src/main/resources/META-INF/persistence.xml**. Define la unidad de persistencia: driver, URL, usuario, proveedor JPA y propiedades de Hibernate.

Propiedades clave de Hibernate

```
hibernate.hbm2ddl.auto = create // Recrea tablas al iniciar (borra datos)
```

```
hibernate.hbm2ddl.auto = update // Actualiza esquema, preserva datos
```

```
hibernate.hbm2ddl.auto = validate // Solo valida, no modifica
```

```
hibernate.show_sql = false // No imprime SQL en consola
```

Borrado Lógico vs Físico

En sistemas reales casi nunca se borra físicamente un registro (**DELETE**). En cambio, se marca como eliminado con un campo booleano:

- **Borrado físico:** `em.remove(entity)` → borra el registro. Puede lanzar excepción si hay foreign keys que lo referencian.
- **Borrado lógico:** `entity.setEliminado(true); em.merge(entity)` → el registro queda pero con `eliminado=true`. Las consultas filtran por `WHERE eliminado=false`.

■ El TP8 tuvo -5 puntos por usar borrado físico (`em.remove`) en un producto que tenía referencias en **DetallePedido** → excepción de integridad referencial.

Patrón Repository

El patrón Repository abstrae el acceso a datos detrás de una interfaz. La lógica de negocio (Main) no sabe cómo se persisten los datos, solo llama métodos como **guardar()**, **buscarPorId()**, etc. Esto hace el código más testeable, mantenible y desacoplado de JPA.

JPAUtil — Singleton del EntityManagerFactory

El **EntityManagerFactory** es costoso de crear (lee persistence.xml, abre la conexión a la BD, etc.). Por eso se crea una sola vez y se reutiliza. JPAUtil lo encapsula como singleton estático.

JPAUtil.java

```
public class JPAUtil {  
    private static final EntityManagerFactory emf =  
        Persistence.createEntityManagerFactory("miUnidad");  
  
    public static EntityManagerFactory getEntityManagerFactory() {  
        return emf;  
    }  
  
    public static void close() {  
        if (emf != null && emf.isOpen()) emf.close();  
    }  
}
```

BaseRepository<T extends Base>

Clase abstracta genérica que implementa las 4 operaciones CRUD comunes a cualquier entidad. El bound **T extends Base** garantiza que T tiene getId() y setEliminado(), evitando cualquier casteo.

Principio clave: cada método crea y cierra su propio EntityManager en un bloque **finally**. Así se evitan EntityManagers "abiertos" o fugas de recursos.

Método	Implementación
guardar(T entity)	em.merge(entity). Sirve para INSERT y UPDATE. Devuelve la entidad con ID generado.
buscarPorId(Long id)	em.find(clase, id). Devuelve Optional.ofNullable() para manejar el caso de no encontrar.
listarActivos()	JPQL: SELECT e FROM e WHERE e.eliminado = false. Usa clase.getSimpleName() como nombre de entidad.
eliminarLogico(Long id)	Busca la entidad, hace setEliminado(true), llama merge(). Devuelve boolean.

BaseRepository.java — estructura completa

```
public abstract class BaseRepository<T extends Base> {

    private final Class<T> clazz;

    private final EntityManagerFactory emf;

    public BaseRepository(Class<T> clazz) {

        this.clazz = clazz;

        this.emf = JPAUtil.getEntityManagerFactory();

    }

    public T guardar(T entity) {

        EntityManager em = emf.createEntityManager();

        try {

            em.getTransaction().begin();

            T merged = em.merge(entity);

            em.getTransaction().commit();

            return merged;

        } catch (Exception e) {

            if (em.getTransaction().isActive())

                em.getTransaction().rollback();

            throw e;

        } finally {

            em.close(); // SIEMPRE se cierra

        }

    }

    public Optional<T> buscarPorId(Long id) {

        EntityManager em = emf.createEntityManager();

        try {

            return Optional.ofNullable(em.find(clazz, id));

        } finally { em.close(); }

    }

    public List<T> listarActivos() {

        EntityManager em = emf.createEntityManager();

        try {

            String jpql = "SELECT e FROM " + clazz.getSimpleName()

                + " e WHERE e.eliminado = false";

            return em.createQuery(jpql, clazz).getResultList();

        } finally { em.close(); }

    }

}
```

```

public boolean eliminarLogico(Long id) {
    EntityManager em = emf.createEntityManager();
    try {
        T entity = em.find(clazz, id);
        if (entity == null) return false;
        em.getTransaction().begin();
        entity.setEliminado(true);
        em.merge(entity);
        em.getTransaction().commit();
        return true;
    } catch (Exception e) {
        if (em.getTransaction().isActive())
            em.getTransaction().rollback();
        throw e;
    } finally { em.close(); }
}
}

```

CategoriaRepository y ProductoRepository

Implementaciones concretas

```

// Solo necesita el constructor con super()
public class CategoriaRepository extends BaseRepository<Categoria> {
    public CategoriaRepository() { super(Categoria.class); }
}

// Agrega buscarPorCategoria con JPQL tipada
public class ProductoRepository extends BaseRepository<Producto> {
    public ProductoRepository() { super(Producto.class); }

    // JPQL: productos activos de una categoria dada
    public List<Producto> buscarPorCategoria(Long categoriaId) {
        EntityManager em = JPAUtil.getEntityManagerFactory().createEntityManager();
        try {
            TypedQuery<Producto> q = em.createQuery(
                "SELECT p FROM Producto p WHERE p.categoria.id = :categoriaId"
                + " AND p.eliminado = false", Producto.class);
            q.setParameter("categoriaId", categoriaId);
            return q.getResultList();
        } finally { em.close(); }
    }
}

```

```
}
```

Errores comunes a evitar — lección del TP8

■ Error: `getSingleResult()` sin try/catch

Si la query no encuentra resultados lanza `NoResultException` y rompe el programa. Solución: usar `find()` con `Optional`, o envolver en try/catch.

■ Error: Borrado físico (`em.remove`) con FKs

Si un `Producto` tiene `DetallePedidos`, removiéndolo se viola la integridad referencial. Solución: borrado lógico siempre.

■ Error: No cerrar `EntityManager`

Si el EM no se cierra en finally, puede haber fugas de conexión. Solución: siempre `em.close()` en el bloque finally.

■ Error: `rollback` sin verificar `isActive()`

Si la transacción no comenzó (el error fue antes del `begin()`), llamar `rollback()` lanza otra excepción. Solución: `if (em.getTransaction().isActive()) em.getTransaction().rollback()`.

GUIÓN DE PRESENTACIÓN — 2DO PARCIAL PROGRAMACIÓN III

El video debe mostrar el proyecto funcionando y explicar las decisiones técnicas. Duración sugerida: **10 a 15 minutos**. Tener el IDE y la terminal listos antes de grabar.

PARTE 1 — Introducción (1-2 min)

Presentación personal y del trabajo

Lo que decís:

"Hola, soy Patricio Sussini. Voy a presentar el 2do parcial de Programación III, que consiste en extender el proyecto JPA del TP8 agregando un patrón Repository genérico y un menú de consola para gestionar Categorías y Productos."

Mostrás:

"La carpeta del proyecto abierta en NetBeans / VS Code. Señalás la estructura de paquetes: entities, enums, util, repository, Main."

PARTE 2 — Estructura del proyecto (2-3 min)

Recorrida por los archivos nuevos

Abrís JPAUtil.java y explicás:

"Este es el singleton del EntityManagerFactory. Se crea una sola vez al cargar la clase porque es costoso de instanciar. Tiene un método close() para liberar la conexión al salir."

Abrís BaseRepository.java y explicás:

"Esta es la clase abstracta genérica. El tipo T está acotado a Base para poder llamar setEliminado() sin castear. Cada método crea su propio EntityManager y lo cierra en el bloque finally, sin importar si hubo error o no."

Destacás guardar():

"Uso merge() y no persist() porque sirve para las dos cosas: insertar objetos nuevos (sin ID) y actualizar existentes (con ID). Si hay excepción, verifico que la transacción esté activa antes de hacer rollback, para no tirar una segunda excepción."

Abrís ProductoRepository.java y explicás buscarPorCategoria():

"Acá uso TypedQuery de Producto sin casteos. La JPQL navega la relación p.categoria.id directamente, sin necesitar un JOIN explícito. El parámetro es nombrado con :categoriaId para mayor claridad."

PARTE 3 — Demo en vivo (5-7 min)

Ejecutar la aplicación y recorrer el menú

Ejecutás con `mvn exec:java` (o desde el IDE) y mostrás:

"El menú principal limpio, sin logs de Hibernate."

Demo Categorías — Alta:

"Creás una categoría, mostrás el mensaje con el ID generado automáticamente."

Demo Categorías — Listado:

"Mostrás la tabla formateada con ID, NOMBRE, DESCRIPCION."

Demo Categorías — Modificación:

"Seleccionás una categoría. Dejá el nombre vacío (Enter) → se mantiene. Cambiás solo la descripción."

Demo Categorías — Baja lógica:

"Das de baja una categoría. Verificás con Listado que ya no aparece. "El registro no se borra de la BD, solo tiene eliminado=true.""

Demo Productos — Alta:

"Mostrás que primero lista las categorías activas. Ingresás datos con validaciones: probás poner precio negativo → mensaje de error. Después ingresás datos válidos."

Demo Productos — Listado:

"Mostrás la tabla con ID, NOMBRE, CATEGORIA, STOCK, PRECIO."

Demo Reportes — Productos por Categoría:

"Acá se ejecuta la JPQL de ProductoRepository. Seleccionamos una categoría y se listan solo los productos activos de esa categoría."

PARTE 4 — Explicación técnica final (1-2 min)

Cierre con los conceptos clave

Explicás:

"En este parcial apliqué tres conceptos centrales del curso:"

1 — JPA y Hibernate:

"Las entidades ya existían del TP8. No las modifiqué. El persistence.xml tiene hbm2ddl.auto=update para que los datos persistan entre ejecuciones."

2 — Patrón Repository genérico:

"BaseRepository centraliza el CRUD evitando repetición de código. CategoriaRepository y ProductoRepository solo aportan lo específico de cada entidad."

3 — Borrado lógico:

"Ninguna entidad se borra físicamente. Esto evita excepciones de integridad referencial y permite recuperar datos si fuera necesario."

Cerrás con:

"Gracias. Patricio Sussini, Programación III, UTN 2026."

Consejos antes de grabar

- Corré el proyecto **una vez antes** de grabar para que la BD ya tenga el esquema creado.
- Prepárate **datos de ejemplo** con nombres y precios reales (no 'aaa', '123').
- Cerrá notificaciones y otras ventanas del sistema operativo.
- Tené los archivos clave abiertos en el IDE antes de empezar: BaseRepository.java, ProductoRepository.java, Main.java.
- No leas el guión literalmente — hablá como si lo explicarás a un compañero.